Clean Street

Description

Clean Street – Civic Issue Reporting & Tracking Application

Clean Street is a full-stack civic engagement platform that enables citizens to report local public issues such as garbage dumps, potholes, water leakages, and broken streetlights. Users can submit complaints with photos and location details, ensuring accurate and transparent reporting.
The system routes complaints to the appropriate authorities or volunteers and allows real-time status tracking. Community members can participate through voting and comments, while administrators monitor activities and generate reports through a dedicated dashboard. The platform promotes accountability, efficiency, and active public participation in maintaining cleaner communities.

Purpose

- To provide a simple and accessible digital platform for reporting civic issues.
- To ensure location-based routing of complaints to responsible authorities.
- To enable real-time tracking of complaint status for transparency.
- To encourage community participation through voting and feedback.
- To assist administrators in monitoring, managing, and analyzing civic issues effectively.

Getting Started

Prerequisites

Before setting up and running the application, ensure the following software and tools are installed on your system:

- Node.js and npm
  Download and install from Node.js official website. npm comes bundled with Node.js.
- MongoDB
  Install and set up MongoDB locally or use a cloud-hosted solution like MongoDB Atlas.
- SMTP Service
  Configure an SMTP service (e.g., Gmail SMTP) for sending OTPs and notifications.
- IDE

Use a development environment like Visual Studio Code for code editing and debugging.

Installation

Follow these steps to set up the project:

Clone the Repository

```
Git clone https://github.com/springboardmentor211/Group2_CleanStreet.git
cd Group2_CleanStreet
```

Install Dependencies

```
npm install
```

Running the Application

Start the backend

Navigate to the backend directory:

cd backend

Run the backend server (ensure the backend is running on a specific port. E.g., 5000)

npm run dev

Start the frontend

Navigate to the frontend directory:

cd frontend

Run the backend server

npm run dev

Accessing the Application

- Frontend: Visit http://localhost:3000 (or the port specified during the setup).
- Backend API: Accessible at http://localhost:1000 for testing endpoints.

Environment Setup for Clean Street

To run the project seamlessly, you need to set up environment variables for both the backend and frontend. Create a .env file in the root directory of the backend project and add the required configurations as shown below.

Required Environment Variables

Variable Name	Description	Example Value
USER_DB_URL	Connection string for the user database.	mongodb://localhost:27017/users
JOB_DB_URL	Connection string for the job postings database.	mongodb://localhost:27017/jobs
NOTIFICATION_DB_URL	Connection string for the notifications database.	mongodb://localhost:27017/notifications
APPLICATION_DB_URL	Connection string for the applications database.	mongodb://localhost:27017/applications
JWT_SECRET	Secret key for signing JWT tokens.	supersecretkey123
SMTP_HOST	SMTP host for sending emails.	smtp.gmail.com
SMTP_PORT	SMTP port for the email server.	587
SMTP_USER	Email address used for sending emails.	your-email@example.com
SMTP_PASS	Password or app-specific password for the email.	your-email-password

Example .env Configuration

Here's an example .env file for your project:

MonoftoDB URLs

USER_DB_URL=mongodb://localhost:27
017/users

JOB_DB_URL=mongodb://localhost:2
7017/jobs
NOTIFICATION_DB_URL=mongodb://localhost:27017/notifications
APPLICATION_DB_URL=mongodb://localhost:27017/applications
JWT Secret
JWT_SECRET=supersecretkey123

SMPT Configuration

SMPT_HOST=smtp@gmail.com

SMTP_PORT=578

SMTP_USER=your

Steps to Setup the Environment

- Create a .env file in the root of your backend directory.
- Copy the example configuration above and paste it into the .env file.
- Replace placeholder values (like your-email@example.com and your-email-password) with your actual configuration.

Additional Notes

- For cloud-hosted MongoDB (e.g., MongoDB Atlas), replace mongodb://localhost:27017/<db> with your cluster's connection string.
- If you're using a third-party email service, adjust the SMTP_HOST, SMTP_PORT, SMTP_USER, and SMTP_PASS accordingly.
- Keep the .env file secure and do not commit it to version control. Add .env to your .gitignore file.

Folder Structure

A well-organized folder structure helps maintain clarity and efficiency in development. Below is a suggested folder structure for Clean Street Project, leveraging the MERN stack.

```
Group2_CleanStreet/
 ├── backend/
|   ├── scripts/ (admin & setup scripts)
|   ├── src/
|   |   ├── config/ (DB, Cloudinary, Mail, Multer, Passport)
|   |   ├── middleware/ (rate limiter)
|   |   ├── models/ (User, Report, Event, Volunteer)
|   |   ├── routes/ (auth, admin, reports, volunteers, setup)
|   |   └── server.js
|   ├── package.json
|   └── .env
 ├── frontend/
|   ├── public/ (static files)
```

```
|   ├── src/
|   |   ├── components/
|   |   ├── contexts/
|   |   ├── pages/ (admin, user, volunteer, auth)
|   |   ├── utils/
|   |   ├── App.js
|   |   └── main.jsx
|   ├── package.json
|   └── .env
├── README.md
├── LICENSE
└── .gitignore
```

Key Features

- **Smart Issue Reporting**
  Users can report civic issues (garbage, drainage, road damage, etc.) with images and location details for quick resolution.
- **Community Interaction System**
  Users can view, comment, and engage with reported issues to promote community participation.
- **Volunteer Registration & Verification**
  Volunteers can register, create profiles, and get admin approval before organizing community events.
- **Event Creation & Management**
  Verified volunteers can create clean-up events, manage participation, and track event progress.
- **Real-Time Status Tracking**
  Users can monitor issue status updates (Pending, In Progress, Resolved) in real time.
- **Admin Dashboard**
  Admins can manage users, verify volunteers, monitor reports, and oversee overall platform activity.
- **Secure Authentication System**
  Role-based authentication (Admin, User, Volunteer) using JWT for secure login and access control.
- **Image Upload Integration**
  Cloud-based image storage for issue reports and event media using Cloudinary.
- **Email Notification System**
  Automated email verification, password reset, and important platform notifications.
- **Responsive UI Design**
  Fully responsive frontend ensuring seamless experience across mobile and desktop devices.

Project Components

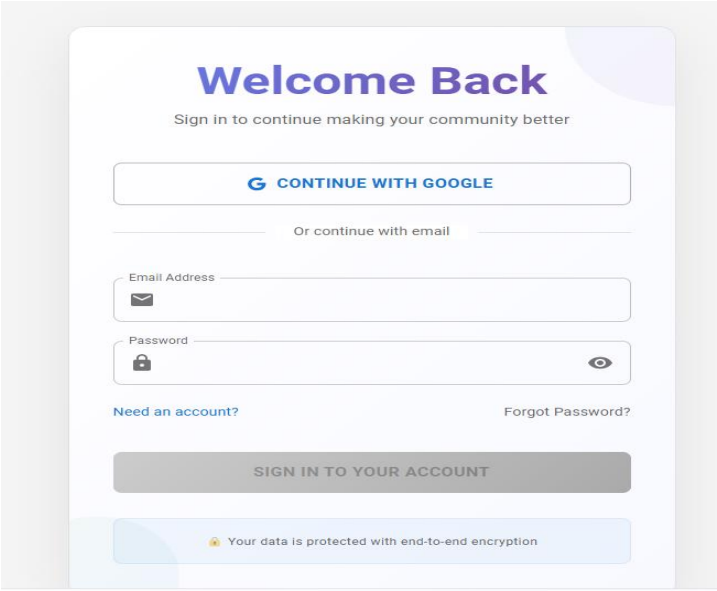
Here's a list of key components in the Clean Street project, along with brief descriptions of their functionality:

1 Login Component

Description:
Handles secure authentication for Admin, User, and Volunteer roles using JWT-based login.

Key Features:

- **Secure Credential Verification:** Validates user email and encrypted password before granting access.
- **JWT Token Generation:** Generates a secure token to maintain authenticated user sessions.
- **Role-Based Redirection:** Redirects users to their respective dashboards after successful login.
- **Session Management:** Ensures protected access to internal pages after authentication.



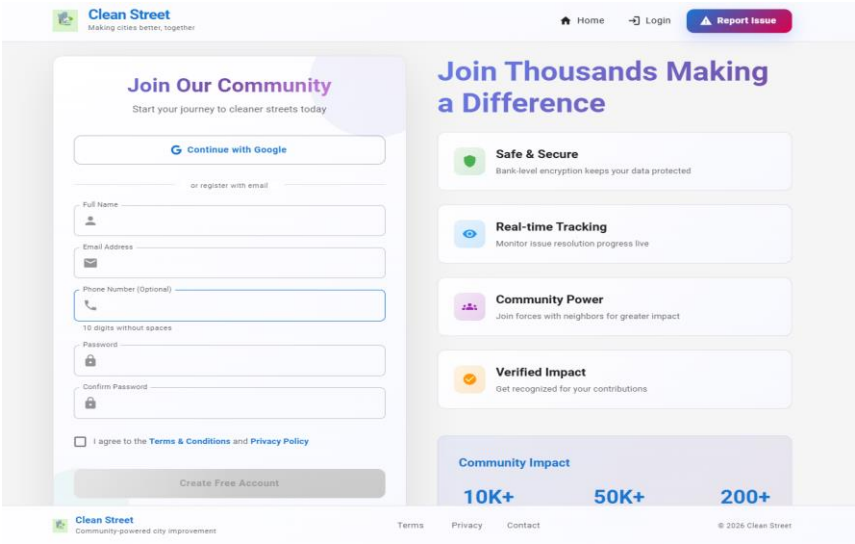
---

2 Register Component

Description:
Allows new users to create accounts with proper validation and email verification.

Key Features:

- **User Data Collection:** Captures essential user details like name, email, and password.
- **Email Verification Process:** Sends verification link to confirm user identity.
- **Input Validation:** Ensures strong password rules and valid email format.
- **Role Selection Support:** Allows registration as User or Volunteer.



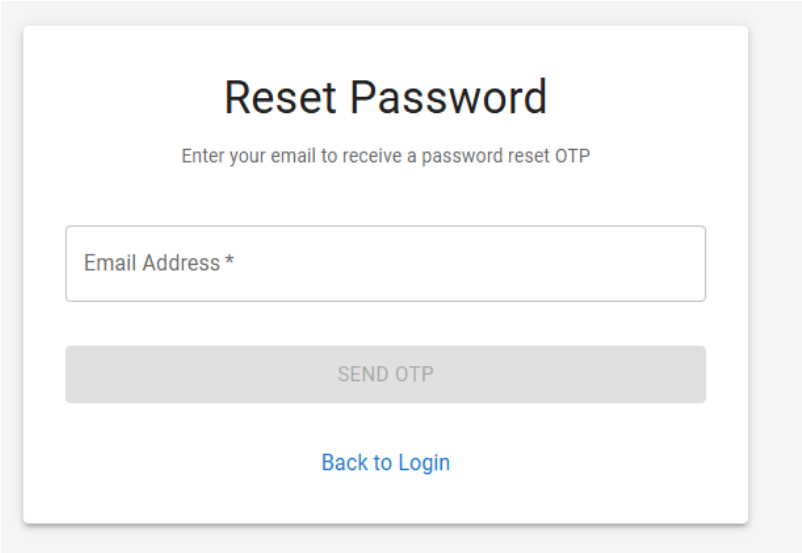
---

3 Reset Password & OTP Component

Description:
Implements password recovery functionality using OTP verification via email.

Key Features:

- **OTP Generation & Delivery:** Generates a secure one-time password and sends it via email.
- **OTP Verification Mechanism:** Validates the OTP entered by the user for identity confirmation.
- **Secure Password Update:** Allows password reset only after successful OTP verification.
- **Time-Limited OTP Expiry:** Ensures OTP is valid only for a limited duration for security.



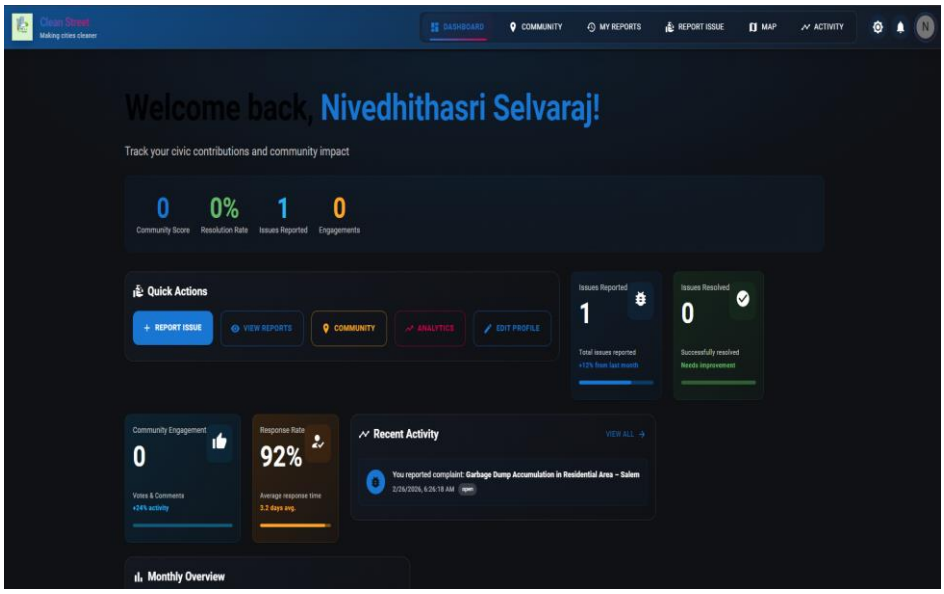
---

4 User Dashboard Component

Description:
The User Dashboard provides a personalized space where users can view and manage their reported issues.

Key Features:

- **Issue Summary**: Displays all reported issues with current status.
- **Status Tracking:** Shows real-time updates (Pending, In Progress, Resolved).
- **Quick Navigation:** Easy access to report new issues and view activity.
- **Responsive Design:** Works smoothly on mobile and desktop.



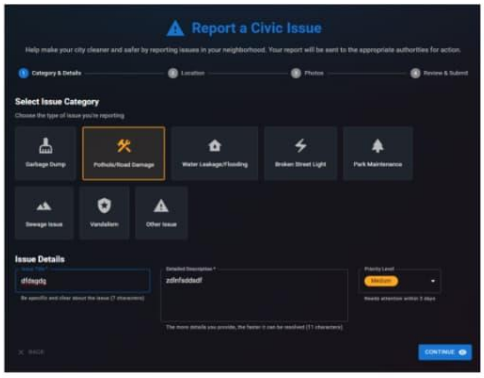
---

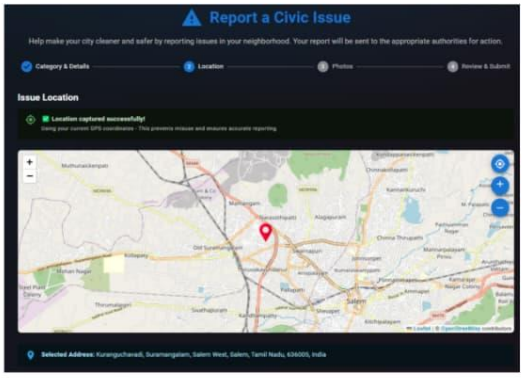
5 Report Issue Component

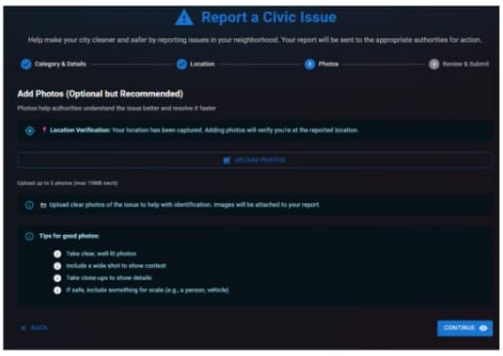
Description:
Allows users to report civic issues with relevant details and supporting images.

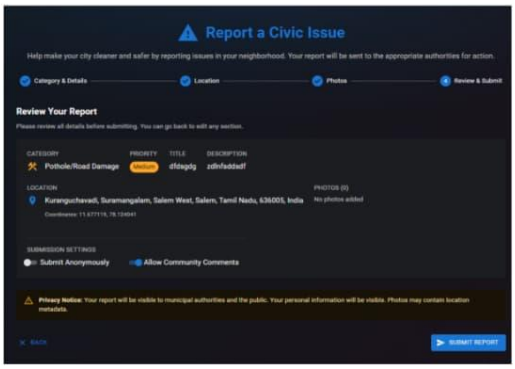
Key Features:

- **Detailed Issue Submission:** Users can provide description, category, and location.
- **Image Upload Integration:** Allows uploading issue images for better clarity.
- **Real-Time Data Storage:** Saves issue data securely in the database.
- **Status Initialization:** Sets issue status to “Pending” upon submission.







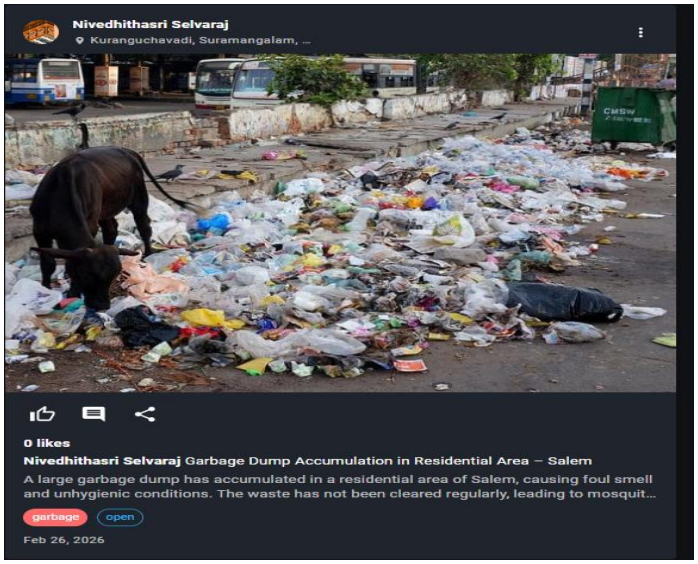


6️⃣ IssueCard Component

Description:
Displays reported issues in an organized card-based layout.

Key Features:

- **Structured Information Display:** Shows issue title, description, and status clearly.
- **Visual Status Indicators:** Uses labels for Pending, In Progress, or Resolved.
- **Interactive Design:** Allows users to view details or comment on issues.

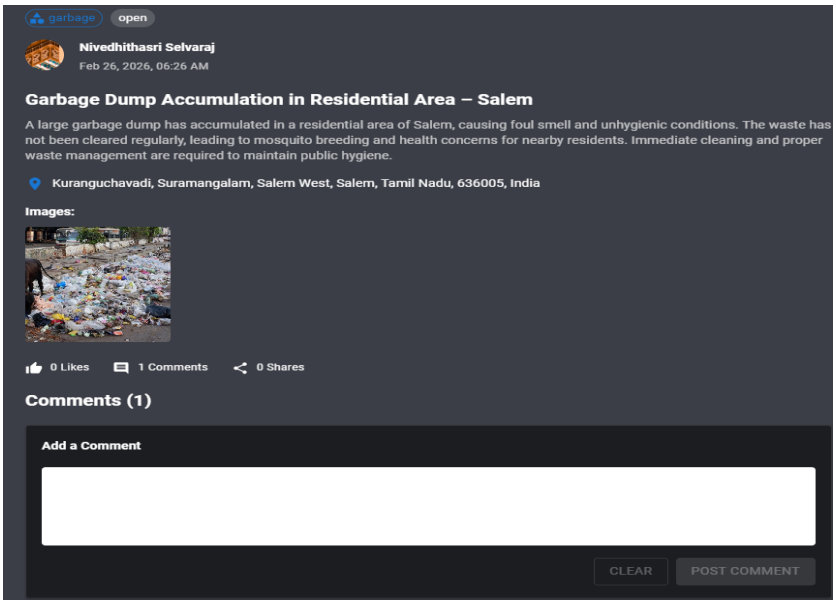


7️⃣ Comment Section Component

Description:
Enables users to interact and discuss reported issues.

Key Features:

- **Add & View Comments:** Allows community members to post feedback.
- **User Attribution:** Displays commenter name and timestamp.
- **Real-Time Interaction:** Updates comments dynamically for better engagement.

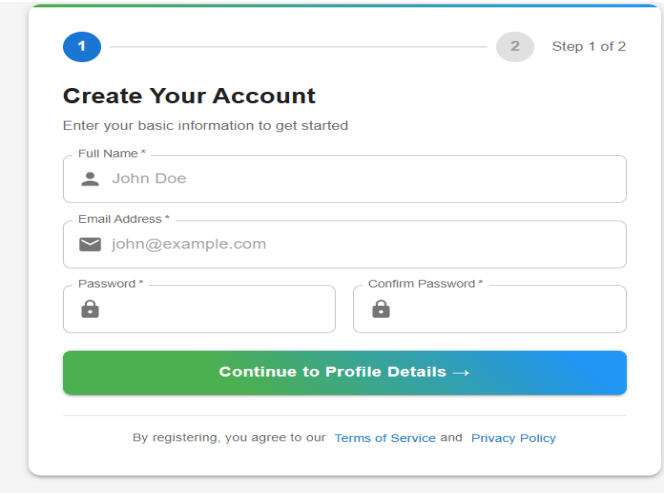


8️⃣ Volunteer Registration Component

Description:
Allows users to apply as volunteers for community activities.

Key Features:

- **Volunteer Application Form:** Collects detailed volunteer information.
- **Admin Approval Workflow:** Applications require admin verification before activation.
- **Status Tracking:** Shows application status (Pending/Approved/Rejected).

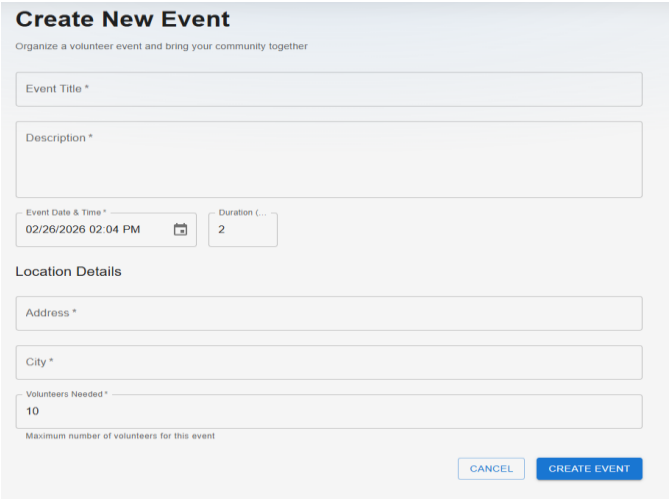


9️⃣ Create Event Component

Description:
Enables approved volunteers to organize community clean-up events.

Key Features:

- **Event Creation Interface:** Allows input of event name, date, and location.
- **Participant Management:** Tracks registered participants for each event.
- **Event Status Control:** Allows monitoring and updating event progress.



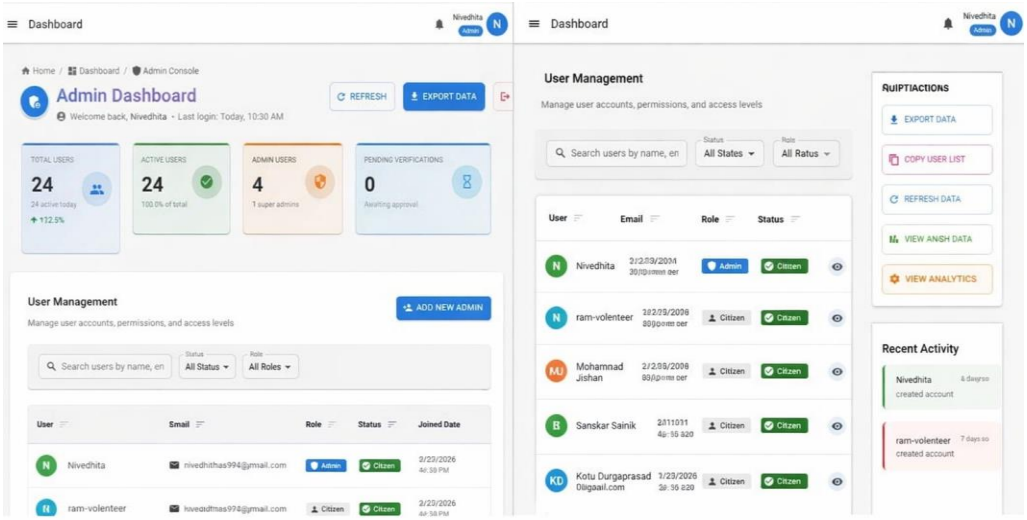
🔟 Admin Dashboard

Description:
Provides centralized administrative control over the entire platform.

Key Features:

- **Platform Analytics Overview:** Displays total users, reports, and events.

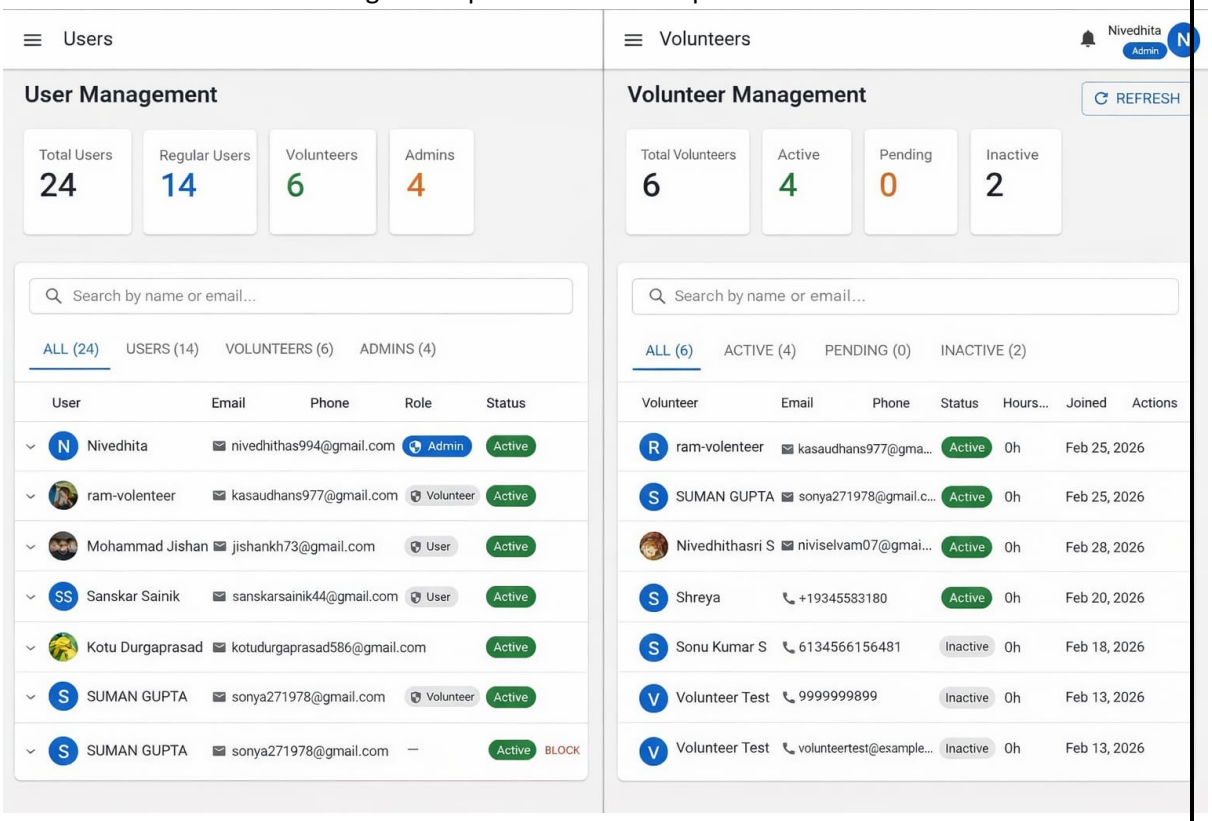
- **User & Volunteer Management:** Enables monitoring and control of accounts.
- **Report Monitoring System:** Allows updating issue statuses efficiently.



---

1 1 Reports Management Component

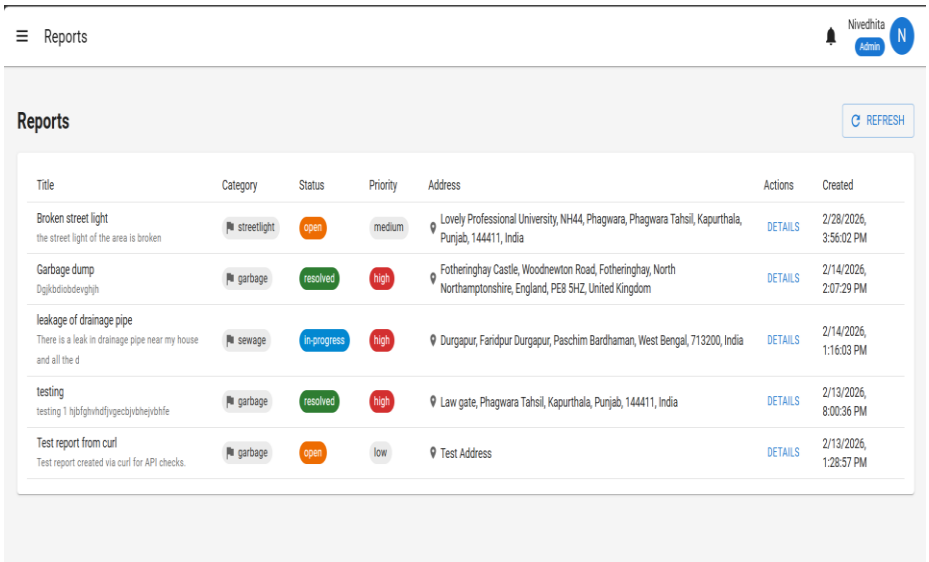
Description:
Allows administrators to manage and update civic issue reports.



Key Features:

- **Status Update Controls:** Change issue status to In Progress or Resolved.

- **Filtering & Sorting Options:** Organize reports for easier monitoring.
- **Administrative Oversight:** Ensures proper handling of community issues.

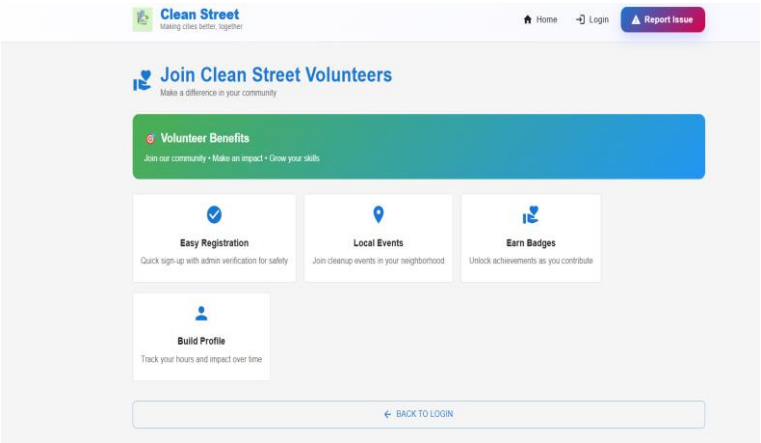


1 3 Volunteer Dashboard

Description:
Provides volunteers with an overview of their events and activities.

Key Features:

- **Event Summary Display:** Shows created and upcoming events.
- **Profile Management:** Allows updating volunteer details.
- **Activity Monitoring:** Tracks participation and engagement metrics.

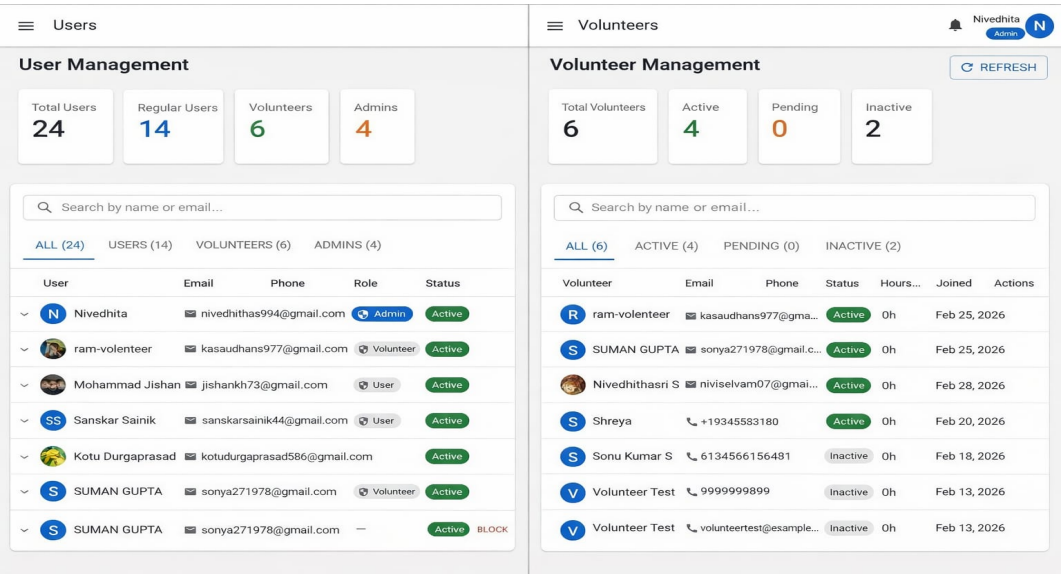


---

1 4 User & Volunteer Management Module

Description: Enables administrators to efficiently manage all user and volunteer accounts within the platform.

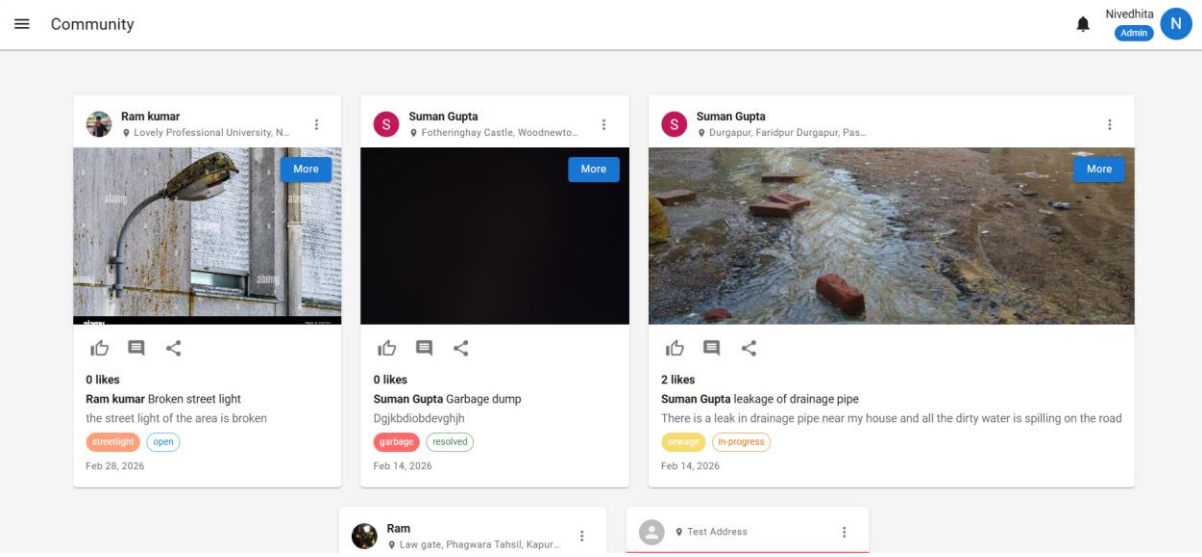
Key Features:
• Account Monitoring: View and track all registered users and volunteers.
• Role Management: Assign, update, or modify user roles and permissions.
• Account Control: Activate, deactivate, or suspend accounts when necessary.
• Profile Oversight: Review and verify user and volunteer information.



1 5 Community Management Module

Description: Allows administrators to supervise and manage overall community interactions and engagement activities.

Key Features:
• Community Posts Monitoring: Review and moderate posts shared within the platform.
• Content Approval System: Approve or remove inappropriate or irrelevant content.
• Engagement Tracking: Monitor discussions, feedback, and community participation.
• Event & Campaign Supervision: Oversee community-driven initiatives and awareness programs.



API Components

Clean Street – Garbage Reporting & Management System

Base URL:
http://localhost:1000

User Authentication & Management

User Registration

Route: POST /signup
Description: Register a new citizen account.

Input:

```
{
  "username": "Krithika R",
  "email": "krithika@gmail.com",
  "password": "Krithika@123"
}
```

Output:
200 OK: { "message": "Signup successful" }
400 Bad Request: { "error": "Invalid input" }

User Login

Route: POST /login
Description: Authenticate a registered user.

Input:

```
{
  "email": "krithika@gmail.com",
  "password": "Krithika@123"
}
```

Output:
200 OK: { "message": "Login successful" }
401 Unauthorized: { "error": "Invalid credentials" }

Send OTP for Password Reset

Route: POST /send-otp
Description: Sends OTP to user email for password reset.

Input:

```
{
  "email": "krithika@gmail.com"
}
```

Output:
200 OK: { "message": "OTP sent successfully" }
400 Bad Request: { "error": "Failed to send OTP" }

Verify OTP

Route: POST /verify-otp
Description: Verify OTP before allowing password reset.

Input:

```
{
  "email": "krithika@gmail.com",
  "otp": "458921"
}
```

Output:
200 OK: { "message": "OTP verified successfully" }
400 Bad Request: { "error": "Invalid OTP" }

Logout

Route: POST /logout
Description: Logs out the current user.

Output:
200 OK: { "message": "Logged out successfully" }

Garbage Complaints Management

Report Garbage

Route: POST /complaints
Description: Allows users to report garbage dumping in a specific location.

Input:

```
{
  "title": "Garbage dump near bus stop",
  "description": "Large pile of waste near T. Nagar bus stand",
  "location": "T. Nagar, Chennai",
  "image": "Image file upload"
}
```

Output:
200 OK: { "message": "Complaint submitted successfully" }
400 Bad Request: { "error": "Invalid complaint data"

Get All Complaints

Route: GET /complaints
Description: Retrieve all reported garbage complaints.

Output:

```
[
  {
    "complaintId": "1",
    "location": "T. Nagar",
    "status": "Pending"
  }
]
```

Get Complaint by ID

Route: GET /complaints/:id
Description: Retrieve a specific complaint.

Output:

```
{
  "complaintId": "1",
  "title": "Garbage near park",
  "status": "Resolved"
}
```

Update Complaint Status (Admin)

Route: PUT /complaints/:id
Description: Update complaint status (Pending / In Progress / Resolved).

Input:

```
{
  "status": "Resolved"
}
```

Output:
200 OK: { "message": "Status updated successfully" }
400 Bad Request: { "error": "Invalid status" }

Delete Complaint

Route: DELETE /complaints/:id
Description: Delete a complaint.

Output:
200 OK: { "message": "Complaint deleted successfully" }
404 Not Found: { "error": "Complaint not found" }

Notifications

Get User Notifications

Route: GET /notifications/:userId
Description: Retrieve notifications for a user.

Output:

```
[
  {
    "notificationId": "12345",
    "message": "Your complaint has been resolved"
  }
]
```

Mark Notification as Read

Route: PUT /notifications/:id/read
Description: Mark notification as read.

Output:
200 OK: { "message": "Notification marked as read" }

Admin Dashboard APIs

Get All Users

Route: GET /admin/users
Description: Retrieve all registered users.

Output:

```
[
  {
    "userId": "1",
    "username": "Krithika R"
  }
]
```

Get Complaint Statistics

Route: GET /admin/stats
Description: Retrieve complaint statistics (total, pending, resolved).

Output:


```
{
  "totalComplaints": 120,
  "pending": 30,
  "resolved": 90
}
```


Error Handling and Troubleshooting

Effective error handling ensures the smooth functioning of the Clean Street – Civic Issue Reporting & Management System. Below are common issues and their solutions.

Gmail SMTP Not Working

Error Description:

OTP emails or notification emails are not being delivered to users.

Troubleshooting Steps:

Ensure the correct Gmail credentials are configured in the environment file. Verify SMTP settings such as smtp.gmail.com with port 587 (TLS). If two-step verification is enabled, generate and use an App Password instead of the regular Gmail password. Also check the spam or junk folder to confirm emails are not blocked.

Port Conflicts

Error Description:

The server fails to start because the configured port (e.g., 1000) is already in use.

Troubleshooting Steps:

Check which process is using the port using system commands like lsof or netstat. Stop the conflicting process or change the port number in the application configuration file and restart the server.

Database Connection Failure

Error Description:

The application is unable to connect to the MongoDB database.

Troubleshooting Steps:

Verify the database connection string and credentials. Ensure that the MongoDB service is running properly. Confirm that the database user has appropriate read and write permissions.

API Request Failures

Error Description:

Frontend requests fail to receive proper responses from backend APIs.

Troubleshooting Steps:
Ensure the correct API endpoint and base URL are used. Verify that authentication tokens (JWT) are properly included in request headers. Check HTTP status codes such as 404 for incorrect routes, 401 for unauthorized access, and 500 for server errors.

File Upload Issues

Error Description:
Users are unable to upload images while submitting civic issue reports.

Troubleshooting Steps:
Ensure the file size is within the allowed limit and the format is supported (JPG or PNG). Check that sufficient storage space is available on the server and that upload middleware is properly configured.

Authentication Issues

Error Description:
Users are logged out unexpectedly or cannot access protected routes.

Troubleshooting Steps:
Check if the JWT token has expired. Ensure the token is stored and sent correctly with each request. Verify that login credentials are valid and correctly processed by the backend.

General Troubleshooting Tips

Clear the browser cache and reload the application. Check server logs for detailed error messages. Use browser developer tools to inspect console errors. Restart the backend server if necessary.

Maintenance and Support

Regular database backups should be scheduled to prevent data loss. Sensitive credentials such as API keys and email passwords should be stored securely in environment variables. Server logs should be monitored regularly to ensure system stability.

Users can report issues through a dedicated support email or contact form available within the system. A help section or FAQ page should be maintained to address common user concerns.

Version Control and Updates

Git should be used for version control to manage code changes efficiently. The main branch should contain production-ready code, while feature branches can be used for development. Updates should be tested in a staging environment before deployment. Database changes should be applied carefully to avoid data inconsistencys.

Glossary

Civic Issue:
Public infrastructure problems such as garbage dumping, potholes, drainage blockage, water leakage, or streetlight failure reported by citizens.

Complaint ID:
A unique number assigned to each complaint for tracking and reference.

Status Tracking:
Shows the progress of a complaint as Pending, In Progress, or Resolved.

Admin Dashboard:
A management panel used by authorities to review, update, and monitor reported issues.

OTP Verification:
A one-time password sent to the registered email for secure password reset.

FAQs

How do I report a civic issue?
Login to your account, go to the Report Issue section, enter the details, upload an image if needed, and submit.

Can I track my complaint status?
Yes, users can view the current status of their complaint in the dashboard.

What types of issues can be reported?
Garbage issues, potholes, drainage problems, water leakage, streetlight faults, and other civic-related concerns.

How can I reset my password?
Use the “Forgot Password” option, verify via OTP, and set a new password.

Why am I not receiving notifications?
Check your spam folder and ensure your registered email address is correct.

Future Enhancements

Mobile Application Support:
Developing a mobile application for easier reporting with real-time GPS location detection.

Map Integration:
Displaying reported issues on a city map for better visualization and tracking.

AI-Based Issue Classification:
Using artificial intelligence to automatically identify and categorize issues from uploaded images.

Advanced Analytics Dashboard:
Providing detailed reports on issue trends, resolution time, and area-wise statistics for better decision-making.